

Build your own SECOND BRAIN

The principles and starter prompts behind the second brain system from the video – so Claude Fable 5 can build one that fits *your* workspace, not copy mine.

The one thing to hold onto:

Every workspace is different. The way my files, folders and memory are organised is not the way yours are – so my exact system would fit you badly. What transfers are the **principles**. Give these to Fable, let it study *your* workspace, and let it build *your* version.

What you need

- Claude Code with Fable 5 – while it's included in the subscription, let it do the heavy lifting.
- A workspace with some history in it – notes, projects, memory files. The brain needs something to organise.
- An hour or two of letting Fable work, checking in between steps.

How this guide works

Five principles, each with a starter prompt you can paste as-is. The prompts deliberately ask Fable to interview you before building – answering its questions is where your workspace's shape gets in.

1 Research before you build

Don't start from your own guesses - or mine. Have Fable pull what's actually working for people *right now*, then read your workspace, and only then plan. The combination of "current best practice" plus "how my files actually look" is what makes the plan fit.

I used the `/last30days` skill by **Matt Van Horn** for the research sweep - it scours Reddit, X, YouTube, Hacker News and more for the last 30 days of real usage. If you don't have it, the prompt below asks for the same thing in plain words.

STARTER PROMPT

```
research how people are building second brain / memory systems for claude code right now - reddit, x, youtube, hacker news, the lot. whats actually working, whats hype. then scan my whole workspace and describe how my files, folders and memory are organised today. combine the two into a short plan for a second brain that fits THIS workspace. before you build anything, ask me at least 3 questions about how i actually work.
```

2 Stand on proven shoulders

Four open-source memory projects are worth studying before building - not to copy, but to steal the right ideas from:

- **Karpathy's LLM wiki** - the foundation: plain markdown the agent writes itself, one small index read first. [gist.github.com/karpathy/442a6bf...](https://gist.github.com/karpathy/442a6bf555914893e9891c11519de94f)
- **qmd** (Tobi Lütke, Shopify's founder) - search by meaning, not just keywords. github.com/tobi/qmd
- **gbrain** (Garry Tan, YC's CEO) - answers cite their sources; the brain cleans itself. github.com/garrytan/gbrain
- **Graphify** (YC-funded) - your notes already form a graph; use the links. github.com/safishamsi/graphify

Link or screenshot them into your prompt and ask Fable to judge what fits - and what to skip.

STARTER PROMPT

```
before building, study these four open source memory projects: karpathy's llm wiki (gist.github.com/karpathy/442a6bf555914893e9891c11519de94f), qmd by tobi lutke (github.com/tobi/qmd), garry tan's gbrain (github.com/garrytan/gbrain), and graphify (github.com/safishamsi/graphify). for each one tell me what idea is worth stealing for MY setup and what to skip - with reasons. dont copy any of them wholesale. then update the build plan. ask me 3 questions if anything about my workspace is unclear.
```

3 Deterministic code before the model

This is where the savings come from. Most of a memory lookup needs **zero intelligence** - so it should be plain code, not model calls. The ladder my system climbs for every question:

- Strip the question down to its keywords - filler words get discarded.
- Score every possible source **without opening any of them** - using the index and file names alone.
- Open only the top-scoring file - not three, not "just in case". One.
- Read only the section that answers - not the whole document.
- If that section just points somewhere else, follow the pointer once.
- **Only then** does the model see anything - it gets the question plus the evidence, and answers in one go.

All of that runs in milliseconds because it's code, not model turns. The model is invoked exactly once, at the end.

STARTER PROMPT

```
build the retrieval path as plain deterministic code, not model calls. it should: strip my question to keywords, score every candidate source from the index WITHOUT opening files, open only the single best file, pull only the section that answers, and follow one pointer if that section points elsewhere. the model only gets involved at the very end, with the evidence already attached. also give me one simple command that stores a new memory - writes the file and updates the index in one step, no model needed.
```

4 Keep a small index of everything

The scoring trick above only works because my workspace keeps **indexes and reference maps** - one small catalogue file where every memory has a one-line entry, and a map of which big doc owns which topic. That's what lets code judge "where would this answer live?" without reading anything.

If you build one habit, build this one: every new fact gets a file *and* a line in the catalogue, every time. An index that's always true is what keeps retrieval cheap forever.

STARTER PROMPT

```
set up a single small index file that catalogues every memory in one line each - name, link, one-sentence description. make saving a new memory automatically add its index line so the catalogue can never drift from reality. then add a short routing note to my claude md so every session knows: check the index first, open files second.
```

5 Make it prove itself

Don't take the system's word for it - make Fable test its own build. Run the same questions through a fresh default session and through the second brain, then compare with **/context** (it shows exactly how many tokens the session burned) and a clock. If the brain isn't clearly cheaper, the instruction is simple: keep optimising until it is.

One honest thing the tests will teach you: for facts that already sit in your always-loaded index, default Claude is instantly fast - that's your index doing its job. The brain wins on the deep stuff: facts buried inside files, questions that need more than one file, and saving new memories.

STARTER PROMPT

now prove the system works. design a fair test: the same real questions from my workspace answered by a fresh default session vs the second brain path. compare tokens with /context and wall time for both sides, and check the answers are actually correct. if the brain doesnt clearly win, keep optimising and retesting until it does. show me the results in a simple table.

The **/GOAL** trick - make Fable check its own speed

The tip that made the biggest difference to quality: give Fable a **/goal** with a hard pass-fail line and permission to keep working until it's met. It will test its own build, catch its own lag, and iterate without you babysitting. I used this for the interactive parts - anything you click, drag or reload.

SAMPLE /GOAL PROMPTS

/goal make sure that moving things around and switching views in the interface never lags for me. actually load it and check your own work - dont just assume. keep optimising until its smooth.

/goal every time i reload this it has to be ready in under ten seconds. verify by reloading it yourself and timing it, and continuously optimise the whole thing until it consistently hits that.

/goal dont stop until the retrieval system is clearly faster and cheaper than default, verified by real tests you run yourself - and have the test prompts ready for me so i can rerun them anytime.

Want the import-ready version?

The full system from the video - ready to import and adapt to your workspace, plus the walkthrough - lives in the RoboNuggets community: skool.com/robonuggets

Fable 5 is a seriously powerful model - and while it's part of the subscription rather than usage-based pricing, it's the perfect moment to offload this kind of build to it. Every workspace is different: let it build yours.